Enterprise Mobile Automation Framework

Comprehensive Technical Documentation

Appium-Based Production-Ready Framework
for Android & iOS Applications

Version: 1.0.0

Date: June 17, 2026

Author: Nishant Pawar

Repository: github.com/nishantpawar90/MobileAutomation

Table of Contents

1. Framework Overview & Architecture

This is a production-ready, enterprise-grade Mobile Automation Framework built on top of Appium, designed to automate both Android and iOS native mobile applications. The framework follows industry best practices including the Page Object Model (POM), Factory Design Pattern, Builder Pattern, Singleton Pattern, and implements thread-safe parallel execution capabilities.

Key Architectural Principles:

- **Separation of Concerns:** Test logic, page interactions, driver management, and configuration are completely decoupled into independent layers.
- **Single Responsibility:** Each class handles exactly one responsibility - DriverFactory creates drivers, DriverManager manages lifecycle, BasePage handles element interactions.
- **Open/Closed Principle:** Framework is extensible (add new pages, new platforms) without modifying existing code.
- **Thread Safety:** All shared state uses ThreadLocal variables enabling safe parallel execution across multiple devices.
- **Configuration-Driven:** Every behavior can be controlled via properties files, system properties, or environment variables without code changes.

Architecture Layers:

Layer	Responsibility	Key Classes
Test Layer	Test cases, assertions, test data	BaseTest, LoginTests, HomePageTests, CheckoutE2ETest
Page Object Layer	Element locators, page interactions	BasePage, LoginPage, ProductCatalogPage, CartPage
Driver Layer	Driver creation, lifecycle, capabilities	DriverFactory, DriverManager, DeviceConfig
Config Layer	Properties, secrets, constants	ConfigManager, SecretManager, FrameworkConstants
Reporting Layer	Screenshots, videos, reports	TestListener, ScreenshotUtils, VideoRecorder
Data Layer	Test data from JSON, Excel, DB, Faker	JsonDataReader, ExcelDataReader, TestDataGenerator
Enterprise Layer	Performance, network logs, self-healing	PerformanceMonitor, NetworkLogCapture, SelfHealingLocator
Utility Layer	Gestures, waits, API client	MobileGestures, WaitUtils, ApiClient

2. Technology Stack & Dependencies

The framework leverages a carefully selected set of modern, actively maintained libraries that together provide a comprehensive mobile test automation solution.

Component	Technology	Version	Purpose
Language	Java	17 (LTS)	Core programming language with modern features (records, sealed classes, pattern matching)
Build Tool	Maven	3.9+	Dependency management, build lifecycle, profiles for different execution modes
Mobile Automation	Appium Java Client	9.3.0	Mobile automation protocol - drives Android/iOS via W3C WebDriver protocol
WebDriver Protocol	Selenium	4.19.1	W3C WebDriver implementation, element interactions, wait strategies
Test Framework	TestNG	7.9.0	Test organization, parameterization, groups, parallel execution, listeners
Android Driver	UiAutomator2	7.6.1	Android automation engine - fast, reliable element identification
iOS Driver	XCUITest	Latest	iOS automation engine - Apple native test framework integration
Reporting (HTML)	Extent Reports	5.1.1	Rich HTML reports with screenshots, categories, dashboard
Reporting (Allure)	Allure TestNG	2.25.0	Detailed step-by-step reports with attachments, history, trends
Logging	Log4j2	2.23.0	Structured logging with console + file appenders, async support
JSON Processing	Jackson	2.16.1	Test data reading, API response parsing, config deserialization
API Testing	REST Assured	5.4.0	Backend API validation alongside mobile UI tests
Database	MongoDB Driver	4.11.1	Test data retrieval from database, state verification
Test Data	DataFaker	2.1.0	Dynamic realistic test data generation (names, addresses, emails)
Excel Data	Apache POI	5.2.5	Excel-based test data reading for data-driven tests
AOP	AspectJ	1.9.21	Allure step annotation weaving at compile time
Cloud Device Farm	BrowserStack	API v2	Real device testing in cloud - 3000+ device/OS combinations

3. Project Structure

The framework follows Maven standard directory layout with clear separation between main source (framework code) and test source (test cases).

```
MobileAutomation/
■■■ .github/workflows/mobile-automation.yml   # GitHub Actions CI/CD pipeline
■■■ pom.xml                                    # Maven build configuration
■■■ testng.xml                                 # Default test suite
■■■ testng-smoke.xml                           # Smoke test suite (7 critical tests)
■■■ testng-regression.xml                      # Full regression suite
■■■ testng-browserstack.xml                    # BrowserStack device matrix
■■■ src/main/java/com/enterprise/mobile/
■   ■■■ api/          # REST Assured API client
■   ■■■ config/       # ConfigManager, FrameworkConstants, SecretManager
■   ■■■ data/         # JsonDataReader, ExcelDataReader, TestDataGenerator, MongoDBClient
■   ■■■ driver/       # DriverFactory, DriverManager, DeviceConfig, BrowserStackCapabilities
■   ■■■ enterprise/   # PerformanceMonitor, NetworkLogCapture, SelfHealingLocator
■   ■■■ enums/        # Platform, ExecutionMode, Environment, SwipeDirection, WaitStrategy
■   ■■■ exceptions/   # Custom exception hierarchy
■   ■■■ listeners/    # TestListener, RetryAnalyzer, AnnotationTransformer
■   ■■■ pages/        # BasePage + android/ + ios/ page objects
■   ■■■ reporting/    # ExtentReportManager, AllureReportManager, ScreenshotUtils
■   ■■■ utils/        # VideoRecorder, WaitUtils, MobileGestures
■■■ src/main/resources/config/              # Properties files per environment
■■■ src/test/java/.../tests/                # Test classes (login, home, e2e, smoke)
■■■ src/test/resources/                     # Test data (JSON), APK files, allure config
```

4. Appium Configuration & Setup

Appium is the core automation engine that enables cross-platform mobile testing. This framework uses Appium 2.x architecture with the java-client 9.3.0, which communicates with automation drivers (UiAutomator2 for Android, XCUITest for iOS) via the W3C WebDriver protocol.

4.1 Appium Server Requirements

- **Appium Server:** Version 2.x+ (installed globally via npm: `npm install -g appium`)
- **UiAutomator2 Driver:** `appium driver install uiautomator2` - Provides Android automation
- **XCUITest Driver:** `appium driver install xcuitest` - Provides iOS automation
- **Server URL:** Default http://127.0.0.1:4723 (configurable in config.properties)
- **Protocol:** W3C WebDriver (Appium 2.x dropped JSONWP/Mobile JSON Wire Protocol)

4.2 Appium Desired Capabilities - Android

The framework configures UiAutomator2Options with the following capabilities for Android testing:

Capability	Value	Purpose
automationName	UiAutomator2	Specifies the automation engine for Android
deviceName	emulator-5554	Target device (emulator or real device serial)
platformVersion	14	Android API level / OS version
app	src/test/resources/apps/SauceLabs-MyDemoApp.apk	Path to the APK under test
appPackage	com.saucelabs.mydemoapp.android	Target app package name
appActivity	...view.activities.SplashActivity	Launch activity of the app
autoGrantPermissions	true	Auto-accept runtime permissions
noReset	false	Do not preserve app state between sessions
fullReset	false	Do not uninstall app between sessions
noSign	true	Skip APK re-signing (avoids apksigner dependency)

4.3 Appium Desired Capabilities - iOS

Capability	Value	Purpose
automationName	XCUITest	Apple native automation engine
deviceName	iPhone 14	Target iOS simulator or real device
platformVersion	16.4	iOS version
app	src/test/resources/apps/ios-app.app	Path to .app or .ipa file
bundleId	com.example.app	App bundle identifier
autoAcceptAlerts	true	Auto-accept system popups and alerts
wdaLocalPort	Auto-assigned	Unique port per device for parallel execution

4.4 Selenium Version Pinning (Critical)

Appium java-client 9.3.0 declares a dependency range [4.19.0, 5.0) for Selenium. Maven resolves this to the latest available (4.45.0+), which removed the ContextAware interface that Appium still uses. The framework uses

<dependencyManagement> to pin selenium-api, selenium-remote-driver, and selenium-support to exactly 4.19.1, preventing ClassNotFoundException at runtime.

5. Mobile Environment Setup

5.1 Android Environment Setup

Prerequisites for local Android testing:

- **Java JDK 17+** - Required for compilation and execution (JAVA_HOME must be set)
- **Android SDK** - Install via Android Studio or command-line tools. Set ANDROID_HOME environment variable
- **Platform Tools** - ADB (Android Debug Bridge) for device communication
- **Build Tools 34.0.0** - Required for APK operations
- **System Image** - android-34;google_apis;x86_64 for emulator
- **Android Emulator** - Create AVD: `avdmanager create avd -n Pixel_6 -k 'system-images;android-34;google_apis;x86_64'`
- **Appium Server** - `npm install -g appium` (requires Node.js 18+)
- **UiAutomator2 Driver** - `appium driver install uiautomator2`

Environment Variables Required:

```
JAVA_HOME=C:\path\to\jdk-17
ANDROID_HOME=C:\Users\<user>\AppData\Local\Android\Sdk
PATH=%ANDROID_HOME%\platform-tools;%ANDROID_HOME%\emulator;%ANDROID_HOME%\build-tools\34.0.0
```

Starting the Environment:

```
# 1. Start Android Emulator
emulator -avd Pixel_6 -no-snapshot-load

# 2. Start Appium Server
appium --address 127.0.0.1 --port 4723

# 3. Verify device connection
adb devices # Should show: emulator-5554 device
```

5.2 iOS Environment Setup

- **macOS** - Required (iOS development only supported on Mac)
- **Xcode 15+** - Includes iOS Simulator and development tools
- **Xcode Command Line Tools** - `xcode-select --install`
- **Carthage/CocoaPods** - WebDriverAgent dependency management
- **WebDriverAgent** - Appium's iOS automation server (auto-installed by xcuitest driver)
- **XCUITest Driver** - `appium driver install xcuitest`
- **ios-deploy** - For real device deployment: `npm install -g ios-deploy`

5.3 Target Application

The framework is configured to test **Sauce Labs My Demo App v2.2.0** - a realistic e-commerce mobile application with product catalog, login, cart, and checkout flows. This provides a production-like testing scenario.

Property	Value
App Name	Sauce Labs My Demo App
Version	2.2.0 (Build 25)

Package	com.saucelabs.mydemoapp.android
Launch Activity	com.saucelabs.mydemoapp.android.view.activities.SplashActivity
APK Size	17.1 MB
Valid Credentials	bob@example.com / 10203040
Features	Product catalog, Login, Cart, Checkout, Navigation drawer

6. Driver Management & Factory Pattern

6.1 DriverFactory (Factory Method Pattern)

The DriverFactory class is the single entry point for creating Appium driver instances. It uses the Factory Method pattern to create the appropriate driver (AndroidDriver or IOSDriver) based on the configured platform and execution mode.

Execution Modes Supported:

- **LOCAL:** Connects to a local Appium server (http://127.0.0.1:4723). Tests run on local emulator/simulator or connected USB device.
- **BROWSERSTACK:** Connects to BrowserStack's cloud hub (hub-cloud.browserstack.com). Tests run on real devices in the cloud.
- **SAUCELABS:** Placeholder for Sauce Labs integration (extensible architecture).

Driver Creation Flow:

```
1. BaseTest.setUp() calls DriverFactory.createDriver()
2. DriverFactory reads platform (ANDROID/IOS) and mode (LOCAL/BROWSERSTACK) from config
3. Creates UiAutomator2Options or XCUITestOptions with capabilities
4. Instantiates AndroidDriver or IOSDriver with server URL + options
5. Configures implicit wait timeout
6. Stores driver in DriverManager (ThreadLocal) for thread-safe access
7. Returns the driver to the test class
```

6.2 DriverManager (ThreadLocal Pattern)

DriverManager uses Java's ThreadLocal to store one AppiumDriver instance per thread. This is critical for parallel execution - each test method running in its own thread gets its own isolated driver session. Methods: setDriver(), getDriver(), removeDriver() (which also calls driver.quit()).

6.3 DeviceConfig (Builder Pattern)

DeviceConfig is a POJO with Builder pattern that encapsulates device-specific parameters for parallel execution. It holds: platform, deviceName, platformVersion, udid, systemPort (Android), wdaLocalPort (iOS). When running tests on multiple devices simultaneously, each device needs unique ports to avoid conflicts.

7. Page Object Model (POM) Implementation

7.1 BasePage - Abstract Foundation

Every page object extends BasePage, which provides all common mobile interactions with built-in wait strategies. BasePage obtains the driver from DriverManager (ThreadLocal), initializes Appium PageFactory, and exposes protected methods for element interaction.

Key BasePage Features:

- **Automatic Wait Strategies:** Every interaction uses explicit waits - CLICKABLE (for clicks), VISIBLE (for text entry/read), PRESENCE (for attribute checks), NONE (immediate)
- **FluentWait Support:** Configurable polling interval (500ms default) with ignored exceptions (NoSuchElementException, StaleElementReferenceException)
- **Platform-Aware:** isAndroid(), isIOS(), platformLocator() for cross-platform page objects
- **Gesture Integration:** Each page has access to MobileGestures (swipe, tap, long press, drag-drop)
- **Keyboard Handling:** hideKeyboard() uses 'mobile: hideKeyboard' Appium command (platform-specific)
- **PageFactory Integration:** AppiumFieldDecorator auto-initializes @FindBy annotated elements

7.2 Android Page Objects

Each page object represents one screen/fragment of the application:

Page Object	Screen	Key Methods	Locator Strategy
ProductCatalogPage	Home/Product listing	getProductTitles(), tapProductByName(), openMenu(), goToCart()	By.id, AccessibilityId
NavigationMenuPage	Side drawer menu	openMenu(), goToLogin(), logout(), goToCatalog()	AccessibilityId, XPath+text
LoginPage	Login form	login(), enterUsername(), enterPassword(), tapLogin()	By.id (resource-id)
ProductDetailPage	Product detail	addToCart(), setQuantity(), getProductPrice()	By.id (resource-id)
CartPage	Shopping cart	isCartEmpty(), getTotalPrice(), proceedToCheckout()	By.id (resource-id)
CheckoutInfoPage	Shipping form	fillShippingAddress(), tapToPayment()	By.id (resource-id)

7.3 Locator Strategies Used

- **By.id (resource-id):** Primary strategy - fastest, most reliable. Example: By.id('com.saucelabs.mydemoapp.android:id/loginBtn')
- **AppiumBy.accessibilityId (content-desc):** Used for elements with content-description. Cross-platform compatible. Example: AppiumBy.accessibilityId('View menu')
- **AppiumBy.xpath:** Fallback for complex scenarios combining text + resource-id. Example: //android.widget.TextView[@text='Log In' and @resource-id='...itemTV']

8. Test Execution Strategy

8.1 Test Lifecycle (TestNG)

The test execution follows TestNG's lifecycle managed by BaseTest and TestListener:

```
@BeforeMethod (BaseTest.setUp) → Creates Appium driver session
@BeforeMethod (TestClass.initPages) → Instantiates page objects
↓
@Test → Executes test logic with page object interactions
↓
@AfterMethod (BaseTest.tearDown) → Calls DriverManager.removeDriver() (quits driver)

Each test gets a FRESH driver session (app reinstalled/reset), ensuring test isolation.
```

8.2 TestNG Suite Configuration

TestNG XML files define which test classes to run, with device parameters passed to BaseTest:

```
<suite name="Smoke Test Suite" verbose="2">
  <test name="Android Smoke Tests">
    <parameter name="platform" value="ANDROID"/>
    <parameter name="deviceName" value="emulator-5554"/>
    <parameter name="platformVersion" value="14"/>
    <groups><run><include name="smoke"/></run></groups>
    <classes>
      <class name="...LoginTests"/>
      <class name="...HomePageTests"/>
      <class name="...CheckoutE2ETest"/>
    </classes>
  </test>
</suite>
```

8.3 Test Groups

Group	Purpose	Tests Included
smoke	Critical path validation (run on every commit)	Login, Catalog loads, Prices displayed, Products visible, E2E purchase
regression	Full feature coverage	All smoke + empty fields, locked user, quantity controls, scroll, multi-product cart
login	Authentication-specific tests	Valid login, invalid login, empty fields, locked user, saved credentials
catalog	Product browsing tests	Catalog loads, products displayed, prices, tap product, scroll
e2e	End-to-end user journeys	Complete purchase flow, multi-product cart, checkout validation

8.4 Maven Execution Commands

```
# Run smoke tests locally on Android
mvn test -Dsurefire.suiteXmlFiles=testng-smoke.xml -Dfw.platform=ANDROID -Dfw.execution.mode=LOCAL

# Run regression tests on BrowserStack
mvn test -Pregression -Dfw.execution.mode=BROWSERSTACK

# Run specific test class
mvn test -Dtest=com.enterprise.mobile.tests.login.LoginTests -Dfw.platform=ANDROID
-Dfw.execution.mode=LOCAL

# Run with Maven profile
mvn test -Psmoke   # Uses testng-smoke.xml
mvn test -Pregression   # Uses testng-regression.xml
mvn test -Pbrowserstack   # Uses testng-browserstack.xml
```

9. BrowserStack Cloud Integration

BrowserStack App Automate enables running tests on 3000+ real device and OS combinations without maintaining a physical device lab. The framework integrates seamlessly with BrowserStack.

9.1 How BrowserStack Integration Works

1. **Authentication:** Username and access key are retrieved from environment variables (BROWSERSTACK_USERNAME, BROWSERSTACK_ACCESS_KEY) via SecretManager

2. **App Upload:** APK/IPA is uploaded to BrowserStack via REST API, returning a bs://app-id URL

3. **Capabilities:** BrowserStackCapabilities class builds the bstack:options map with device name, OS version, project/build/session names, and feature flags (video, network logs, debug)

4. **Connection:** DriverFactory creates the driver with URL: https://USERNAME:ACCESS_KEY@hub-cloud.browserstack.com/wd/hub

5. **Execution:** Tests run on real devices in BrowserStack cloud, same test code as local

6. **Results:** Video recordings, network logs, and device logs are available in BrowserStack dashboard

9.2 BrowserStack Capabilities Configuration

Capability	Value	Purpose
userName	From env: BROWSERSTACK_USERNAME	Authentication
accessKey	From env: BROWSERSTACK_ACCESS_KEY	Authentication
projectName	Mobile Automation	Organizes runs in BrowserStack dashboard
buildName	Build-{timestamp}	Groups test sessions into a build
sessionName	Test-{thread-name}	Identifies individual test sessions
debug	true	Enables text logs on BrowserStack
networkLogs	true	Captures HTTP network traffic
video	true	Records video of test execution
appiumVersion	2.4.1	Specifies Appium version on BrowserStack
idleTimeout	300	Max idle time (seconds) before session termination
deviceName	Google Pixel 6 / iPhone 14	Target real device model
osVersion	13.0 / 16	Target OS version

9.3 Security - Secret Management

Credentials are NEVER stored in source code or properties files. The SecretManager resolves secrets in priority order: (1) Environment variables (set in CI/CD), (2) JVM system properties (-DBROWSERSTACK_USERNAME=xxx), (3) Encrypted local store (for development). In GitHub Actions, secrets are configured as repository secrets and injected as environment variables.

10. Reporting & Evidence Capture

10.1 Multi-Layer Reporting

- **Allure Reports:** Step-by-step execution with @Step annotations, attachments (screenshots, page source, logs), history trends, severity/priority categorization. Deployed to GitHub Pages automatically.
- **Extent Reports:** Rich HTML dashboard with pie charts, test timelines, category views. Screenshots embedded inline on failure. Report saved to test-output/reports/.
- **TestNG Reports:** Standard XML/HTML reports in target/surefire-reports/. Used for CI/CD pass/fail determination.
- **Console Logging:** Structured Log4j2 output with test name, timestamps, and context. Logs saved to logs/framework.log.

10.2 Screenshot Capture

Screenshots are captured automatically on every test failure via TestListener.onTestFailure(). Three formats are generated simultaneously: (1) PNG file saved to test-output/screenshots/, (2) Base64 string embedded in Extent Report, (3) byte[] attached to Allure Report. Manual capture is also available via ScreenshotUtils.captureScreenshot(testName).

10.3 Video Recording

Video recording uses Appium's native screen recording APIs (startRecordingScreen / stopRecordingScreen). For Android: 1280x720 resolution, 5Mbps bitrate, 10-minute max. For iOS: Medium quality, 10-minute max. Videos are Base64-decoded and saved as MP4 files in test-output/videos/. On failure, video filename includes '_FAILED' suffix for easy identification. Controlled by config: video.recording=true.

10.4 Performance Monitoring

PerformanceMonitor tracks execution time for every test method (startTimer/stopTimer). Additionally, it can capture Android device metrics: memory usage, CPU utilization, and battery consumption via Appium's 'mobile: getPerformanceData' command. All metrics are attached to Allure reports for performance regression detection.

11. Data-Driven Testing

11.1 JSON Test Data (Jackson)

Test data is stored in src/test/resources/testdata/ as JSON files. JsonDataReader uses Jackson ObjectMapper to deserialize into Maps, Lists, or POJOs. Example: login/valid-credentials.json contains {"username": "bob@example.com", "password": "10203040"}. Tests load data via JsonDataReader.readDataAsMap("login/valid-credentials.json").

11.2 Dynamic Data Generation (DataFaker)

TestDataGenerator wraps the DataFaker library to produce realistic random data for each test run. This prevents test flakiness from hardcoded values and increases coverage. Available generators: generateEmail(), generateFullName(), generateStreetAddress(), generateCity(), generateZipCode(), generateCountry(), generatePhoneNumber(), generatePassword(). Used in checkout E2E tests for shipping address generation.

11.3 Excel Data (Apache POI)

ExcelDataReader supports .xlsx files for large data-driven test suites. Reads sheets into List<Map<String, String>> for TestNG DataProvider integration. Useful for parameterized test scenarios with many combinations.

11.4 MongoDB Integration

MongoDBClient provides connection to MongoDB for: (1) Retrieving test data from collections, (2) Verifying backend state after mobile actions, (3) Setting up test preconditions. Connection string is configured in properties; database name is configurable per environment.

12. Enterprise Features

12.1 Self-Healing Locators

SelfHealingLocator implements automatic locator recovery. When a primary locator fails (element not found), it tries alternative locators (by different strategy - id, accessibilityId, xpath, className). Successful alternatives are cached in a ConcurrentHashMap for the session duration. A healing report is generated at suite end showing all healed locators, enabling proactive maintenance.

12.2 Network Log Capture

NetworkLogCapture records HTTP network activity during test execution using Appium's performance logging. Captured logs include timestamps, request/response data, and are attached to Allure reports on failure. This helps diagnose issues caused by API failures or slow backend responses.

12.3 Mobile Gestures (W3C Actions API)

MobileGestures implements all common mobile interactions using the W3C Actions API (compatible with Appium 2.x). This replaces the deprecated TouchAction/MultiTouchAction APIs.

- **tap(x, y) / tap(element):** Single finger tap at coordinates or element center
- **doubleTap(x, y) / doubleTap(element):** Two rapid taps for zoom or selection
- **longPress(element, durationMs):** Press and hold for context menus
- **swipe(direction):** Swipe UP/DOWN/LEFT/RIGHT with configurable edge offset (20%) and duration (800ms)
- **scrollToElement(element, direction, maxAttempts):** Scroll until element becomes visible
- **scrollWithinElement(container, direction):** Scroll inside a specific scrollable container
- **dragAndDrop(source, target):** Drag from one element to another
- **pinchZoom(scale):** Two-finger pinch gesture for zoom in/out

12.4 API Validation Layer

ApiClient (REST Assured) enables hybrid testing: validate backend state alongside mobile UI tests. Example: After adding a product to cart via mobile UI, verify the cart API returns the correct item. Supports GET, POST, PUT, DELETE with authentication token management.

13. GitHub Actions CI/CD Integration

The framework uses GitHub Actions for continuous integration and delivery. The workflow is defined in .github/workflows/mobile-automation.yml and provides automated test execution on every commit.

13.1 Trigger Conditions

- **Push to main/develop:** Automatically runs smoke tests on BrowserStack
- **Pull Request to main:** Validates changes before merge with full smoke suite
- **Manual Dispatch:** UI-triggered with selectable Platform, Environment, Test Suite, and Execution Mode

13.2 Workflow Steps

1. **Checkout Code** - actions/checkout@v4
2. **Setup JDK 17** - actions/setup-java@v4 with Temurin distribution + Maven cache
3. **Upload App to BrowserStack** - Uploads APK via BrowserStack REST API, captures app_url
4. **Run Tests** - Executes mvn test with platform, environment, mode, and suite parameters
5. **Generate Allure Report** - Builds HTML report from allure-results
6. **Deploy to GitHub Pages** - Publishes Allure report to gh-pages branch (accessible via URL)
7. **Upload Artifacts** - Stores test-output (reports, screenshots, logs) for 30 days
8. **Upload Videos** - Stores failure videos for 7 days (only on failure)
9. **Slack Notification** - Sends pass/fail notification with platform, device, suite, and branch info

13.3 Device Matrix Strategy

The workflow uses a matrix strategy to run tests on multiple device configurations in parallel:

Platform	Device	OS Version
ANDROID	Google Pixel 6	13.0
IOS	iPhone 14	16

13.4 GitHub Secrets Required

Secret Name	Purpose
BROWSERSTACK_USERNAME	BrowserStack account username for API authentication
BROWSERSTACK_ACCESS_KEY	BrowserStack access key (from browserstack.com/accounts/settings)
SLACK_WEBHOOK_URL	Slack incoming webhook URL for notifications
GITHUB_TOKEN	Auto-provided by GitHub for Pages deployment

14. Configuration Management

The ConfigManager implements a layered configuration system with override priority:

```
System Properties (-Dfw.xxx) > Environment Config (qa.properties) > Base Config (config.properties) > Defaults
```

14.1 Configuration Files

File	Purpose
config.properties	Base configuration - platform, timeouts, Appium URL, device caps, app paths
qa.properties	QA environment overrides (default active environment)
dev.properties	Development environment overrides
stage.properties	Staging environment overrides
prod.properties	Production environment overrides

14.2 System Property Overrides

Any property can be overridden at runtime via -Dfw.{key}={value}. The 'fw.' prefix is stripped and the value replaces the file-based property. Common overrides:

```
-Dfw.platform=ANDROID|IOS
-Dfw.execution.mode=LOCAL|BROWSERSTACK|SAUCELABS
-Dfw.environment=QA|DEV|STAGE|PROD
-Dfw.browserstack.app.url=bs://xxxxxx
```

15. Running the Framework - Complete Guide

15.1 Local Execution (Step by Step)

```
# Step 1: Start Android Emulator
emulator -avd Pixel_6

# Step 2: Start Appium Server
appium

# Step 3: Run Smoke Tests
mvn test -Dsurefire.suiteXmlFiles=testng-smoke.xml -Dfw.platform=ANDROID -Dfw.execution.mode=LOCAL

# Step 4: View Reports
# Open: test-output/reports/ExtentReport.html
# Or generate Allure: mvn allure:serve
```

15.2 BrowserStack Execution

```
# Set credentials as environment variables
set BROWSERSTACK_USERNAME=your_username
set BROWSERSTACK_ACCESS_KEY=your_access_key

# Run on BrowserStack
mvn test -Pbrowserstack -Dfw.execution.mode=BROWSERSTACK -Dfw.browserstack.app.url=bs://app-id
```

15.3 Test Results Summary (Current)

The smoke suite (7 tests) executes successfully against the Sauce Labs My Demo App on Android 14 emulator:

Test	Status	Duration	Category
testInvalidLogin	PASSED	~5s	smoke, login
testLoginPageElements	PASSED	~0.5s	smoke, login
testSuccessfulLogin	PASSED	~5s	smoke, login
testProductCatalogLoads	PASSED	~1.4s	smoke, catalog
testProductPricesDisplayed	PASSED	~1.3s	smoke, catalog
testProductsAreDisplayed	PASSED	~2.2s	smoke, catalog
testCompletePurchaseFlow	PASSED	~40s	smoke, e2e

Result: 7/7 tests PASSED | 0 failures | 0 skipped | Total time: ~160 seconds

16. Changing the Test Device (e.g., Pixel 6 to Samsung Galaxy)

The framework supports running tests on any Android or iOS device — physical or virtual. Below is a complete guide to switch from the default Pixel 6 emulator to a different device such as a Samsung Galaxy S23.

16.1 Local Execution - Changing the Android Emulator

Step 1: Create a New AVD (Android Virtual Device)

```
# List available system images
sdkmanager --list | findstr system-images

# Install a system image (if not already installed)
sdkmanager "system-images;android-34;google_apis;x86_64"

# Create a Samsung-like AVD (use any name you prefer)
avdmanager create avd -n Samsung_Galaxy_S23 -k "system-images;android-34;google_apis;x86_64" -d
"pixel_6"

# Or create via Android Studio: Tools > Device Manager > Create Device > Phone > Galaxy S23
# Select a system image > Finish
```

Step 2: Start the New Emulator

```
# List available AVDs
emulator -list-avds

# Start the Samsung emulator
emulator -avd Samsung_Galaxy_S23
```

Step 3: Update Configuration

You have THREE options to change the device. Choose one:

Option A: Update config.properties (permanent change)

```
# File: src/main/resources/config/config.properties
android.device.name=emulator-5554    # Keep as emulator-5554 (ADB serial, same for all emulators)
android.platform.version=14          # Update if changing Android version
```

Option B: Pass as Maven system property (per-run override)

```
mvn test -Dsuite.xml=testng-smoke.xml \
  -Dfw.platform=ANDROID \
  -Dfw.execution.mode=LOCAL \
  -Dfw.android.device.name=emulator-5554 \
  -Dfw.android.platform.version=14
```

Option C: Update TestNG XML parameters (suite-level)

```
<test name="Samsung Galaxy S23 Tests">
  <parameter name="platform" value="ANDROID"/>
  <parameter name="deviceName" value="emulator-5554"/>
  <parameter name="platformVersion" value="14"/>
  <classes>...</classes>
</test>
```

16.2 Physical Device - USB Connected Samsung

To run on a real physical Samsung device connected via USB:

- **Enable Developer Options:** Settings > About Phone > Tap 'Build Number' 7 times

- **Enable USB Debugging:** Settings > Developer Options > USB Debugging = ON
- **Connect via USB** and accept the 'Allow USB debugging' prompt on device
- **Verify connection:** Run `adb devices` — device should show with a serial like 'R5CR1234567'
- **Update config.properties:** Set `android.device.name=R5CR1234567` (use actual serial from adb devices)
- **Run tests:** Same Maven command, device will be targeted automatically

16.3 BrowserStack - Changing Cloud Device

For BrowserStack cloud execution, change the device in the workflow or via Maven properties:

Device Name (BrowserStack)	OS Version	Maven Property Override
Samsung Galaxy S23	13.0	-Dfw.browserstack.device="Samsung Galaxy S23" -Dfw.browserstack.os.version=13.0
Samsung Galaxy S24 Ultra	14.0	-Dfw.browserstack.device="Samsung Galaxy S24 Ultra" -Dfw.browserstack.os.version=14.0
Samsung Galaxy A54	13.0	-Dfw.browserstack.device="Samsung Galaxy A54" -Dfw.browserstack.os.version=13.0
Google Pixel 8	14.0	-Dfw.browserstack.device="Google Pixel 8" -Dfw.browserstack.os.version=14.0
OnePlus 12	14.0	-Dfw.browserstack.device="OnePlus 12" -Dfw.browserstack.os.version=14.0
iPhone 15 Pro	17	-Dfw.browserstack.device="iPhone 15 Pro" -Dfw.browserstack.os.version=17

To update the GitHub Actions matrix for a different device, edit `.github/workflows/mobile-automation.yml` under the `browserstack-test` job's matrix section:

```
strategy:
  matrix:
    include:
      - platform: ANDROID
        device: "Samsung Galaxy S23"  # ← Change device here
        os_version: "13.0"             # ← Change OS version
      - platform: IOS
        device: "iPhone 15 Pro"
        os_version: "17"
```

16.4 Multi-Device Parallel Execution

To run tests simultaneously on multiple devices, add multiple <test> blocks in a TestNG XML file, each with different device parameters. Each test block gets its own thread with isolated driver:

```
<suite name="Multi-Device" parallel="tests" thread-count="3">
  <test name="Samsung Galaxy S23">
    <parameter name="platform" value="ANDROID"/>
    <parameter name="deviceName" value="emulator-5554"/>
    <parameter name="platformVersion" value="14"/>
    <classes>...</classes>
  </test>
  <test name="Pixel 8">
    <parameter name="platform" value="ANDROID"/>
    <parameter name="deviceName" value="emulator-5556"/>
    <parameter name="platformVersion" value="14"/>
    <classes>...</classes>
  </test>
</suite>
```

Note: Each emulator must use a different port (5554, 5556, 5558...). Start multiple emulators with: `emulator -avd Samsung_Galaxy_S23 -port 5556`

17. Configuring & Running Tests on BrowserStack

BrowserStack App Automate allows you to run mobile tests on 3000+ real devices in the cloud. This section provides step-by-step instructions to configure and execute tests on BrowserStack.

17.1 Prerequisites

- **BrowserStack Account:** Sign up at `https://www.browserstack.com/` (free trial available with 100 min)
- **Username & Access Key:** Found at `https://www.browserstack.com/accounts/settings`
- **App uploaded to BrowserStack:** Upload your APK/IPA to get a `bs://` app URL

17.2 Step 1: Upload Your App to BrowserStack

Upload your APK file using cURL (or the BrowserStack dashboard):

```
curl -u "YOUR_USERNAME:YOUR_ACCESS_KEY" \
  -X POST "https://api-cloud.browserstack.com/app-automate/upload" \
  -F "file=@src/test/resources/apps/SauceLabs-MyDemoApp.apk" \
  -F "custom_id=MyDemoApp"

# Response:
# {"app_url": "bs://a1b2c3d4e5f6...", "custom_id": "MyDemoApp"}
# Save the app_url — you'll need it to run tests
```

17.3 Step 2: Set Environment Variables

Set credentials as environment variables (NEVER hardcode in source code):

```
# Windows (Command Prompt)
set BROWSERSTACK_USERNAME=your_username
set BROWSERSTACK_ACCESS_KEY=your_access_key

# Windows (PowerShell)
$env:BROWSERSTACK_USERNAME = "your_username"
$env:BROWSERSTACK_ACCESS_KEY = "your_access_key"

# Linux/macOS
export BROWSERSTACK_USERNAME=your_username
export BROWSERSTACK_ACCESS_KEY=your_access_key
```

17.4 Step 3: Run Tests on BrowserStack

Execute the Maven command with BrowserStack execution mode:

```
# Run smoke tests on BrowserStack (uses default device from testng-browserstack.xml)
mvn test -Pbrowserstack -Dfw.execution.mode=BROWSERSTACK \
  -Dfw.browserstack.app.url=bs://a1b2c3d4e5f6

# Run on a specific device
mvn test -Pbrowserstack -Dfw.execution.mode=BROWSERSTACK \
  -Dfw.browserstack.app.url=bs://a1b2c3d4e5f6 \
  -Dfw.browserstack.device="Samsung Galaxy S23" \
  -Dfw.browserstack.os.version=13.0

# Run regression suite with 3 parallel threads
mvn test -Pregression -Dfw.execution.mode=BROWSERSTACK \
  -Dfw.browserstack.app.url=bs://a1b2c3d4e5f6 \
  -Dthread.count=3
```

17.5 Step 4: View Results on BrowserStack Dashboard

After test execution, view results at `https://app-automate.browserstack.com/dashboard`. Each test session includes:

- **Video Recording:** Full video replay of test execution on the real device
- **Network Logs:** HTTP request/response capture during the test
- **Device Logs:** Logcat (Android) or Console logs (iOS)
- **Screenshots:** Captured at each Appium command
- **Appium Logs:** Complete Appium server logs for debugging
- **Text Logs:** Custom debug logs from the framework

17.6 Step 5: Configure GitHub Actions for BrowserStack

To run BrowserStack tests automatically in CI/CD, configure GitHub repository secrets:

```
1. Go to your GitHub repository: Settings > Secrets and variables > Actions
2. Click 'New repository secret' and add:
    • Name: BROWSERSTACK_USERNAME   Value: your_username
    • Name: BROWSERSTACK_ACCESS_KEY   Value: your_access_key
3. (Optional) Add: SLACK_WEBHOOK_URL for test notifications
4. Go to Actions tab > 'Mobile Automation CI/CD' > 'Run workflow'
5. Select: Execution Mode = BROWSERSTACK, choose Platform and Test Suite
6. Click 'Run workflow' — tests will execute on real cloud devices
```

17.7 How the Framework Connects to BrowserStack (Internal Flow)

When execution.mode=BROWSERSTACK, the following happens internally:

1. **ConfigManager** reads execution.mode=BROWSERSTACK from system property
2. **DriverFactory.createDriver()** detects BROWSERSTACK mode
3. **SecretManager** retrieves username/key from env vars (BROWSERSTACK_USERNAME, BROWSERSTACK_ACCESS_KEY)
4. **BrowserStackCapabilities** builds the bstack:options map with device, OS, project name, video=true, networkLogs=true
5. **DriverFactory** constructs URL: https://username:key@hub-cloud.browserstack.com/wd/hub
6. **AndroidDriver/IOSDriver** is created with the remote BrowserStack URL + capabilities
7. Tests execute identically to local — same Page Objects, same assertions
8. On completion, video/logs are available in BrowserStack dashboard

17.8 BrowserStack Configuration Properties

These properties in config.properties control BrowserStack behavior:

```
# BrowserStack Connection
browserstack.url=https://hub-cloud.browserstack.com/wd/hub
browserstack.app.url=bs://app-id    # Override with -Dfw.browserstack.app.url=bs://xxx
browserstack.build.name=Mobile-Automation-Build
browserstack.project.name=Enterprise-Mobile-Framework

# Credentials (via environment variables - NEVER in this file)
# BROWSERSTACK_USERNAME=your_username
# BROWSERSTACK_ACCESS_KEY=your_access_key

# BrowserStack Features (set in BrowserStackCapabilities.java)
# debug=true          # Enable text logs
# networkLogs=true    # Capture HTTP traffic
# video=true          # Record video
```

```
# appiumVersion=2.4.1 # Appium version on BrowserStack
# idleTimeout=300     # Max idle seconds before kill
```

17.9 Supported BrowserStack Devices

Browse the full device list at: `https://www.browserstack.com/list-of-browsers-and-platforms/app_automate`

Device	Platform	OS Versions Available
Samsung Galaxy S24 Ultra	Android	14.0
Samsung Galaxy S23	Android	13.0, 14.0
Samsung Galaxy A54	Android	13.0
Google Pixel 8 Pro	Android	14.0
Google Pixel 7	Android	13.0
OnePlus 12	Android	14.0
iPhone 15 Pro Max	iOS	17
iPhone 14	iOS	16
iPad Pro 12.9 2022	iOS	16